pontIA

Calidad del Aire Accesible: Diseño e Implementación de un Asistente Conversacional basado en Machine Learning y LLM para la Interpretación de Datos de Contaminación en Madrid

Trabajo Fin de Máster

Máster en Inteligencia Artificial, Cloud Computing y DevOps

AUTORES:
Guillermo Parés Marín
Carlos Fernández Ruiz
Sheila Pena

Tutor/a: Carlos Moreno Morera

CURSO ACADÉMICO: 2026

Índice

Resumen....3
Capítulo 1 - Introducción y caso de negocio....1
1.1 Objetivos.... 1
1.2 Caso de negocio....2
Capítulo 2 - Metodología.... 3
2.1 Organización y metodología de trabajo....3
2.2 Arquitectura del sistema y su evolución....5
2.3 Datos: fuentes, análisis exploratorio y calidad....6
2.4 Pipeline de ingesta y preprocesamiento.... 7
2.5 Detección de anomalías....9
2.6 Predicción....10
2.7 Asistente conversacional.... 11
2.8 Frontend....12
2.9 Observabilidad y métricas.... 13
2.10 Infraestructura en AWS y CI/CD.... 14
2.11 Stack tecnológico....15
Capítulo 3 - Resultados.... 16
3.1 Detección de anomalías....16
3.2 Predicción....16
3.3 Recuperación del RAG....17
3.4 Evaluación del asistente.... 18
3.5 Sistema en producción y costes....19

Resumen

La contaminación atmosférica es uno de los principales riesgos ambientales para la salud en las grandes ciudades. El Ayuntamiento de Madrid publica como datos abiertos las mediciones horarias de su red de 24 estaciones de control, pero en un formato técnico (tablas con una columna por hora, códigos de validación y unidades de concentración) difícil de interpretar para una persona sin formación específica.

Este trabajo presenta el diseño, la implementación y el despliegue en la nube de un sistema que permite consultar en lenguaje natural la calidad del aire de la ciudad de Madrid para seis contaminantes con relevancia sanitaria: NO, NO_2, NOx, O_3, PM10 y PM2.5. El sistema se compone de cinco piezas. La primera es un pipeline de ingesta que descarga los datos de la API municipal al cierre de cada franja del día, los limpia y los almacena en PostgreSQL. La segunda es un detector de anomalías basado en un Isolation Forest por contaminante, que identifica sobre todo fallos de sensor: estaciones que dejan de medir o que registran valores congelados... La tercera es un modelo de predicción a uno y dos días basado en gradient boosting. La cuarta es un asistente conversacional en el que un LLM consulta las mediciones mediante SQL validado de solo lectura y responde a preguntas de salud y normativa con un sistema RAG cuyas citas se comprueban en el backend. La quinta es una aplicación web con estadísticas, un mapa de estaciones y el asistente integrado. Todo el sistema se despliega en Amazon Web Services (Lambda, RDS, S3, EC2 y Amazon Bedrock), con integración continua y despliegue automatizado mediante GitHub Actions..

El detector identifica 10.550 bloques anómalos que un umbral estadístico clásico (z-score) no detecta, en buena parte caídas o congelaciones de sensores. El modelo de predicción supera a la climatología en los seis contaminantes y a la persistencia en cinco de los seis, evaluado sobre un año completo fuera de muestra. Con el umbral de evidencia elegido, el sistema RAG rechaza las 10 preguntas ajenas al dominio del conjunto de evaluación y entrega al LLM la sección correcta en 13 de 15 preguntas documentales. Los resultados muestran que combinar datos abiertos, aprendizaje automático y modelos de lenguaje permite acercar la información medioambiental al ciudadano sin renunciar a la trazabilidad de las respuestas.

El sistema está desplegado y accesible en http://madrid-respira.duckdns.org/

Palabras clave: calidad del aire, detección de anomalías, predicción, LLM, RAG, machine learning, AWS, datos abiertos

Capítulo 1 - Introducción y caso de negocio

La calidad del aire es uno de los factores ambientales con mayor impacto sobre la salud pública. La exposición continuada a contaminantes como el dióxido de nitrógeno (NO_2), el ozono troposférico (O_3) o las partículas en suspensión (PM10 y PM2.5) se asocia a enfermedades respiratorias y cardiovasculares, y sus efectos son mayores en los grupos sensibles: niños, personas mayores y personas con asma o enfermedades cardíacas. Las grandes ciudades, por su densidad de tráfico y de población, son los entornos donde este riesgo se concentra. Madrid, con más de tres millones de habitantes, registra episodios de contaminación por NO_2 que han llevado al Ayuntamiento a aprobar un protocolo de actuación con medidas como la restricción del tráfico.

Las administraciones han hecho en los últimos años un esfuerzo notable por abrir estos datos. El Ayuntamiento de Madrid mantiene una red de 24 estaciones de control que mide de forma continua los principales contaminantes y publica sus mediciones en su portal de datos abiertos, tanto el histórico como los valores del día en tiempo real. Sin embargo, que el dato esté disponible no garantiza que sea útil. Los registros se publican en tablas con una columna por hora, códigos de validación y concentraciones en microgramos por metro cúbico, un formato pensado para técnicos. Existe, por tanto, una brecha entre lo que la administración publica y lo que un ciudadano es capaz de interpretar y usar para tomar decisiones en su día a día.

1.1 Objetivos

El objetivo general de este trabajo es desarrollar y desplegar en la nube un sistema de inteligencia artificial que permita a cualquier persona consultar y comprender, en lenguaje natural, el estado de la calidad del aire en la ciudad de Madrid, combinando datos abiertos en tiempo real, modelos de aprendizaje automático y un asistente conversacional basado en un LLM.

Para alcanzarlo se plantean los siguientes objetivos específicos:

1. Construir un pipeline de ingesta automatizado que cargue el histórico del Ayuntamiento y los datos de la API en tiempo real al cierre de cada franja del día, los limpie y los almacene en una base de datos PostgreSQL que sirva como fuente única de verdad para el resto del sistema.
2. Desarrollar un modelo de detección de anomalías que identifique tanto niveles de contaminación inusuales para cada estación, contaminante y época del año como fallos operativos de los sensores, es decir, estaciones que dejan de medir o que registran valores congelados.
3. Desarrollar un modelo de predicción a corto plazo que estime los niveles de cada contaminante para el día en curso y el siguiente, y medir su aportación frente a referencias simples.

4. Integrar un asistente conversacional basado en un LLM que responda preguntas en lenguaje natural consultando las mediciones, las anomalías y las predicciones, y que fundamente las respuestas sobre salud y normativa en un corpus documental con citas verificables.
5. Desarrollar una aplicación web que muestre el estado de la red de estaciones y dé acceso al asistente.
6. Desplegar el sistema en Amazon Web Services con integración continua y despliegue automatizado, de forma reproducible y con un coste controlado.

1.2 Caso de negocio

Oportunidad y usuarios

La brecha descrita entre el dato publicado y la decisión cotidiana tiene un coste sanitario medible: la Agencia Europea de Medio Ambiente atribuye a la exposición prolongada por encima de los niveles recomendados por la OMS 13.317 muertes anuales en España por PM2.5 y 4.135 por NO_2 (datos de 2023), y Madrid encabeza el ranking europeo de mortalidad atribuible al NO_2, con unas 1.700 muertes prematuras anuales evitables si se cumplieran las guías de la OMS (EEA, 2025; Khomenko et al., 2021).

El banco de unas 50 preguntas reales propuestas por el equipo y recogido al inicio del proyecto permite identificar a qué usuarios se dirige el sistema y qué problemas resuelve: la población sensible y sus familias («¿En qué zona de Madrid está el aire más seguro ahora mismo para una persona con asma?»), deportistas y trabajadores al aire libre («¿Es seguro salir a correr al Retiro en las próximas dos horas?»), personas que planifican eventos o deciden dónde vivir («Si quiero minimizar mi exposición a largo plazo al NO_2, ¿dónde debería buscar vivienda?») y, como beneficiaria indirecta, la administración, cuyo dato abierto se convierte en un servicio ciudadano.

Análisis de mercado y diferenciación

Existen alternativas consolidadas, pero todas resuelven la visualización del dato, no su interpretación (Tabla 1.1).

Alternativa	Qué ofrece	Qué no ofrecía
Portal y app oficiales del Ayuntamiento («Aire de Madrid», Madrid 360)	Datos por estación, mapa de la red, predicción para hoy y mañana y avisos en episodios.	Consulta en lenguaje natural, aviso de fallos de sensor y respuestas de salud con citas verificadas.
Agregadores privados (IQAir, aqicn.org)	Índices AQI en tiempo real, pronóstico a varios días y aplicaciones multiidioma.	Umbrales OMS/UE, concentraciones oficiales de toda la red municipal y validación de sensores.

Capa de calidad del aire de Google Maps	Índice agregado horario sobre el mapa.	Datos por estación y contaminante y recomendaciones de salud detalladas.

[Tabla 1.1. Alternativas existentes y su cobertura]

Ninguna ofrecía una interfaz conversacional sobre el dato oficial, señalización de fallos de sensor ni recomendaciones con citas comprobadas. El diferencial de este sistema no está en una pieza aislada, sino en la combinación: preguntas en lenguaje natural sobre las mediciones oficiales completas, detección de anomalías y fallos de sensor, respuestas de salud y normativa con fuentes verificadas en el backend y predicción propia a uno y dos días.

Viabilidad e impacto

La viabilidad económica se apoya en costes medidos durante el desarrollo y en una estimación de la configuración de entrega. Operar el sistema completo costó entre 14,1 y 17,7 dólares al mes durante el desarrollo, y la configuración de entrega se estima en unos 56 dólares al mes más el uso del modelo de lenguaje.

El coste crece de forma lineal con el uso y, acorde con los costes estimados, mil preguntas cuestan entre 1 y 3 dólares, que se suman al principal componente del gasto, que es la infraestructura base. A esta escala, el sistema es sostenible como servicio público sin ánimo de lucro.

El beneficio esperado es ante todo social: decisiones cotidianas mejor informadas para la población sensible. Su adopción es medible mediante los indicadores de uso y satisfacción integrados en el propio sistema.

Como posibles modelos de explotación se identifican, sin contrastar con el mercado, un convenio institucional con el Ayuntamiento o Madrid Salud, una API de pago por uso para terceros (medios y aplicaciones de deporte y salud) y la replicación en otras ciudades, ya que el pipeline solo depende del formato de datos abiertos municipal.

Capítulo 2 - Metodología

Este capítulo describe cómo se ha construido el sistema: la organización del trabajo, la arquitectura general y cada uno de sus componentes, desde los datos hasta el despliegue en la nube. Para cada decisión relevante se indican también las alternativas que se valoraron y el motivo por el que se descartaron.

2.1 Organización y metodología de trabajo

El desarrollo avanzó en fases incrementales (datos y detección de anomalías, base de conocimiento documental, asistente con LLM e interfaz web), y a mitad del proyecto se fijó un producto mínimo viable para acotar el alcance.

El código vive en un repositorio de GitHub con un flujo de ramas sencillo: cada tarea se desarrolla en una rama propia, se integra en la rama de desarrollo mediante pull request, y solo esta rama puede integrarse en la principal. En cada una, GitHub Actions ejecuta las pruebas automáticas de todos los componentes, incluidas pruebas de integración contra una base de datos PostgreSQL efímera, y comprueba que el contrato entre la API y la web no ha cambiado en un lado sin actualizar el otro. Los componentes que dependen de servicios externos se prueban con dobles: el agente usa un LLM simulado que sigue un guión y un RAG fingido, de modo que sus pruebas no necesitan red ni generan coste.

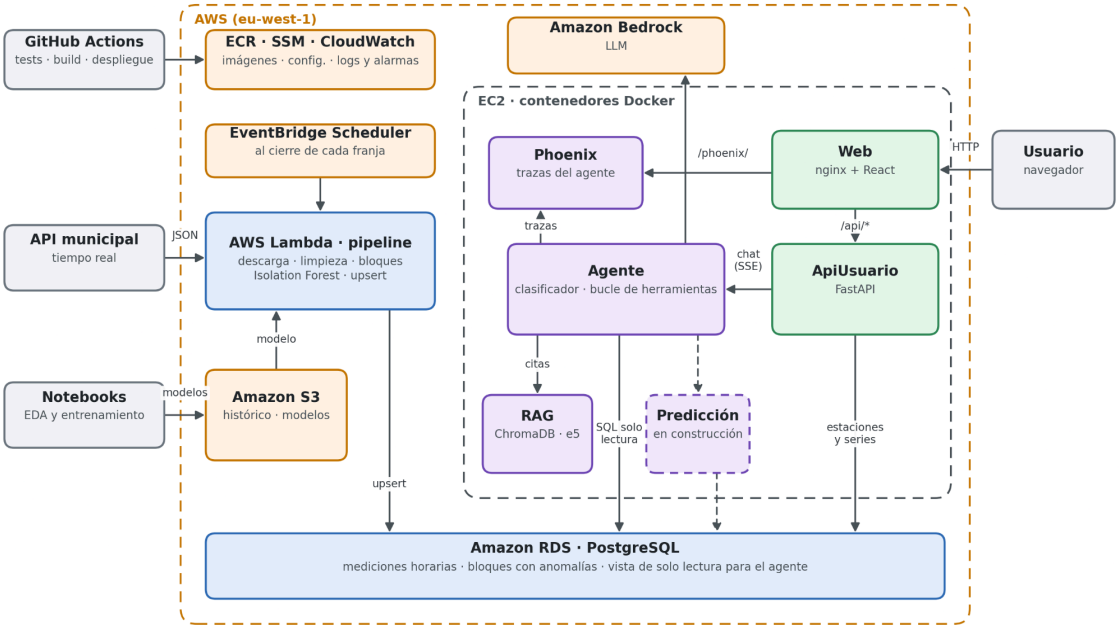
Distribución por miembros del equipo:

- Carlos Fernández se encargó de la estructura del asistente conversacional. Rehízo el sistema RAG como un servicio de evidencias que comprueba las citas del modelo y midió su recuperación para fijar el umbral de evidencia. Sobre él construyó el agente: el clasificador de intención, la tabla de decisión en código, el bucle de herramientas, la memoria de la conversación, la respuesta del modelo, un mecanismo de seguridad para evitar que el modelo imprima el system prompt y las comprobaciones posteriores sobre lo que redacta el modelo. También diseñó el sistema de evaluación de los modelos de Amazon Bedrock con un golden set de evaluacion. Diseñó además la consulta de mediciones: la vista y el usuario de solo lectura en PostgreSQL y las dos variantes de la herramienta, el SQL libre con su validador y el catálogo cerrado. Por último, montó el sistema de trazas y métricas. Instrumentó el agente con OpenTelemetry y Phoenix, que desplegó en la EC2, y llevó los registros, las alarmas y el panel a CloudWatch. Definió también las métricas de negocio del pipeline y los indicadores de uso del asistente, con la valoración de los usuarios y la página de administración de la web.
- Sheila Pena se ocupó de la capa de usuario y de la predicción. Implementó ApiUsuario, la API ligera que proporciona los datos de estaciones y series y conecta con el asistente. Además, desarrolló la aplicación web completa: las pantallas, el mapa de estaciones, el asistente integrado y la identidad visual, junto con los tipos generados a partir del contrato OpenAPI que verifica la CI. En la parte de predicción a corto plazo, trabajó en las variables, los dos horizontes y la evaluación frente a tres referencias, evitando fugas de información. También creó el servicio HTTP que expone la herramienta y evaluó cómo la utilizaban los cuatro modelos candidatos con un conjunto propio de casos de prueba.
- Guillermo Parés se encargó de los datos, del detector de anomalías y de la infraestructura en la nube. Hizo el análisis exploratorio del histórico y escribió el parser único que comparten los notebooks y el pipeline. Definió los bloques del día y el modelo de datos en PostgreSQL. Sobre ellos entrenó el Isolation Forest. Construyó el pipeline de ingesta en tiempo real. Migró el sistema a AWS: el histórico y los modelos a S3, la base de datos a RDS y el pipeline a Lambda, programado con EventBridge al cierre de cada franja del día. Montó la integración y el despliegue continuo: el acceso de GitHub Actions a AWS sin claves, mediante OIDC, y los workflows de despliegue y rollback por componente. También preparó los

contenedores y la EC2, que arranca sola a partir de la configuración guardada en S3 y Systems Manager, y la publicó con una dirección fija y su subdominio, con un presupuesto diario que vigila el gasto en Bedrock. En el asistente, amplió la comparación de modelos con Qwen3 Next y Nemotron 3 Super y llevó Nemotron a producción.

2.2 Arquitectura del sistema y su evolución

El sistema se organiza en tres capas: ingesta y almacenamiento de los datos, servicios de inteligencia (modelos y asistente) y presentación. La Figura 2.1 muestra sus componentes y cómo se comunican.



[Figura 2.1. Arquitectura del sistema desplegado en AWS]

La capa de datos la forma un pipeline que se ejecuta de forma programada en AWS Lambda. En cada ejecución descarga de la API municipal las mediciones del día, las limpia, las agrega en bloques del día, aplica el detector de anomalías y guarda el resultado en una base de datos PostgreSQL gestionada (Amazon RDS). El histórico, los conjuntos de datos procesados y los modelos entrenados se guardan versionados en Amazon S3, que es la fuente común para todo el equipo.

La capa de inteligencia la componen tres servicios HTTP independientes. El servicio RAG busca en el corpus documental de salud y normativa y verifica las citas de las respuestas. El servicio de predicción estima los niveles del día en curso y del siguiente. El agente recibe la pregunta del usuario, decide qué herramientas necesita (consulta de mediciones en SQL, búsqueda documental o predicción), llama al LLM a través de Amazon Bedrock y compone la respuesta.

La capa de presentación es una aplicación web servida por nginx, que además hace de proxy hacia una API ligera llamada ApiUsuario. Esta API lee de la base de datos las estaciones y sus series temporales y reenvía las preguntas del chat al agente, devolviéndo la respuesta. En producción, la web, la API, el agente, los servicios de RAG y predicción y Phoenix, la herramienta que registra las trazas del agente, se ejecutan como contenedores Docker en una instancia EC2 en la que solo nginx expone un puerto al exterior.

El diseño se apoya en cuatro principios:

- Cada dato en el almacén adecuado. Las mediciones se guardan estructuradas en SQL, donde se consultan con exactitud, y la base de datos vectorial guarda solo conocimiento en forma de texto: salud y normativa.
- El modelo de lenguaje propone y el código decide. El LLM clasifica la pregunta y redacta la respuesta, pero los periodos de cada consulta se calculan en código, el SQL que genera se valida antes de ejecutarse con un usuario de solo lectura y sus citas se comprueban contra el corpus.
- Servicios separados por dependencias. Las librerías pesadas (el modelo de embeddings y la base vectorial, de más de 1 GB, o scikit-learn con los modelos de predicción) quedan en su propio servicio. Así el agente y la API son ligeros y cada pieza se despliega por separado.
- El mismo código en entrenamiento y en producción. El parser de los datos y el cálculo de las variables son módulos compartidos por los notebooks y el pipeline, lo que evita discrepancias entre cómo se entrena un modelo y cómo se usa.

2.3 Datos: fuentes, análisis exploratorio y calidad

Los datos proceden del portal de datos abiertos del Ayuntamiento de Madrid. Se utilizan las tres fuentes de la Tabla 2.1.

Fuente	**Contenido**	**Uso**
Histórico de calidad del aire (CSV, 78 MB)	Mediciones horarias del 1 de enero de 2018 al 30 de abril de 2026	Análisis exploratorio, entrenamiento de los modelos y valores de referencia
API en tiempo real (Ciudades Abiertas)	Mediciones horarias del día en curso, actualizadas cada 20 minutos aproximadamente y con alrededor de una hora de retraso	Ingesta continua
Catálogo de estaciones	24 estaciones con tipo, dirección, coordenadas y contaminantes que miden	Mapa, filtros y consultas por zona

[Tabla 2.1]

El histórico y la API comparten un formato ancho: cada fila corresponde a una estación, una magnitud y un día, con 24 columnas de valores (H01 a H24) y 24 indicadores de validación (V01 a V24). El catálogo no incluye el distrito de cada estación, así que se asignó a mano a partir de su dirección para poder responder preguntas por distrito; cuatro estaciones situadas en el límite entre dos distritos quedaron marcadas para revisión.

El histórico contiene 417.629 filas de estación, magnitud y día, con 24 estaciones y 14 magnitudes. El proyecto se centra en seis contaminantes con relevancia sanitaria y cobertura suficiente: NO, NO_2, NOx, O_3, PM10 y PM2.5. Con ellos quedan 320.129 filas y 7.574.463 mediciones horarias válidas.

Del análisis exploratorio salieron varias decisiones que condicionan el resto del sistema:

- El indicador de validación N no marca mediciones dudosas, sino huecos rellenados. Alrededor del 45 % de esas horas valen cero y el resto incluye valores físicamente imposibles, como 89.337 µg/m³ o −42 µg/m³ de PM10. Por eso las horas marcadas con N se tratan como dato ausente y nunca como anomalía.
- La cobertura es desigual: NO, NO_2 y NOx se miden en las 24 estaciones, pero PM10 solo en 15 y PM2.5 en 11. Esto llevó a entrenar un modelo por contaminante y a comparar cada estación con su propio nivel habitual.
- Los óxidos de nitrógeno muestran dos picos diarios, que coinciden con las horas punta del tráfico, y el ozono alcanza su máximo por la tarde por su origen fotoquímico. Una media diaria diluiría esos picos; por eso el sistema trabaja con cuatro bloques del día (apartado 2.4).
- La estacionalidad es opuesta: el ozono es máximo en verano y los óxidos de nitrógeno en invierno. El valor esperado de una medición debe depender, al menos, de la estación, el contaminante, el bloque del día y el mes.
- Los datos recogen episodios reales. El día con la media de PM10 más alta del histórico fue el 15 de marzo de 2022, con 337,5 µg/m³ de media y picos horarios de 721 µg/m³ en la estación de Vallecas, coincidiendo con el episodio de polvo sahariano de esas fechas. La distribución de PM10 tiene una cola derecha larga: la mediana es de 14 µg/m³, el percentil 99 de 76 y el percentil 99,9 de 157.

2.4 Pipeline de ingesta y preprocesamiento

El pipeline transforma los datos de la fuente, publicados en formato ancho con una columna por hora, en dos niveles de detalle: mediciones horarias y bloques del día enriquecidos con estadísticos y con la salida del detector de anomalías. El histórico y los datos en tiempo real pasan por el mismo código.

Transformación. Un parser único convierte cada fila del formato ancho en 24 filas, una por hora, y asocia a cada valor su indicador de validación. Las horas que no están marcadas como válidas, incluidas las marcadas con N, se guardan como nulas, de modo que un hueco nunca se confunde con una medición de cero. Los notebooks de análisis y entrenamiento y el pipeline en tiempo real comparten este parser y el módulo que calcula las variables del

modelo. Así se evita la discrepancia entre entrenamiento y producción (training-serving skew), un fallo habitual en sistemas de aprendizaje automático en el que el modelo recibe en producción datos preparados de forma distinta a como se entrenó.

Bloques del día. Las mediciones horarias se agregan en cuatro bloques por estación, contaminante y día (Tabla 2.2). Un bloque conserva el comportamiento de cada franja sin diluir los picos, como haría una media diaria, y reduce cada día de 24 valores a 4 filas. Para cada bloque se calculan la media, la desviación típica, el número de horas válidas y la cobertura (horas válidas sobre horas del bloque), y se compara con su valor esperado: la media y la dispersión históricas para la misma estación, contaminante, bloque y mes, 5.376 combinaciones en total. El histórico completo da lugar a 1.274.644 bloques. Como los bloques no duran lo mismo, la media de un día o de un distrito se calcula ponderando por el número de horas y no promediando los bloques directamente.

Bloque	**Horas**	**Duración**	**Justificación**
Madrugada	00–06	7 h	Nivel de fondo
Mañana	07–12	6 h	Hora punta del tráfico y picos de NO_2
Tarde	13–19	7 h	Máximo diario de O_3
Noche	20–23	4 h	Tráfico de la tarde-noche

[Tabla 2.2]

Modelo de datos. La base de datos PostgreSQL guarda cuatro tablas y una vista:

- **calidad_aire_horas_live:** mediciones horarias en formato largo, con una clave única por estación, contaminante, fecha y hora.
- **resumen_datos_ml:** una fila por estación, contaminante, día y bloque, con los estadísticos, el valor esperado y la salida del detector. Es la tabla principal del sistema: la API de la web la consulta directamente y el asistente, solo a través de la vista mediciones_bloques.
- **baseline_historico:** valor esperado y dispersión por estación, contaminante, bloque y mes.
- **estaciones**: las 24 estaciones con su tipo, coordenadas, contaminantes medidos y distrito.
- **mediciones_bloques:** vista que une los bloques con la estación y el distrito y se limita a los dos últimos años. Es lo único que puede leer el agente, mediante un usuario sin permisos sobre las tablas.

La carga inicial del histórico usa el comando COPY de PostgreSQL, ya que la inserción fila a fila no escalaba a más de un millón de filas.

Pipeline en tiempo real. En cada ejecución, el pipeline:

1. Descarga las mediciones del día de la API municipal, con reintentos, porque la API devuelve a veces respuestas vacías
2. Las transforma con el mismo parser que el histórico.
3. Las carga en una tabla temporal y desde ella en la tabla horaria.
4. Agrega el día en bloques, calcula las variables y aplica el detector de anomalías.
5. Inserta o actualiza en resumen_datos_ml los bloques del día que ya han terminado; los bloques en curso no se escriben.

La propiedad central del pipeline es la idempotencia: repetir una ejecución no duplica datos, de modo que puede programarse con cualquier frecuencia sin coordinar ejecuciones. Conseguirla obligó a corregir un error sutil. La API devuelve siempre las 24 horas del día, y las que todavía no se han medido llegan como ceros marcados con N. La primera versión ignoraba los registros ya existentes (ON CONFLICT DO NOTHING), así que la primera ejecución del día fijaba esos ceros y ninguna posterior los corregía. Se sustituyó por una actualización condicionada: una hora guardada solo se sobrescribe si la medición entrante es válida y la guardada no.

2.5 Detección de anomalías

El detector tiene que funcionar sin datos etiquetados, ya que no existe un registro oficial de qué mediciones fueron anómalas. Por eso se plantea como un problema no supervisado: identificar los bloques que se apartan del comportamiento habitual de su estación y contaminante. Se distinguen dos familias de anomalías, ambientales y operativas, y cada una deja una señal distinta en los datos (Tabla 2.3).

Familia	**Ejemplo**	**Variable**
Ambiental	Ozono inusualmente alto para un mes de julio	Z_SCORE: desviación respecto al valor esperado, en unidades de dispersión histórica
Operativa: sensor caído	Faltan horas en el bloque	COBERTURA: horas válidas sobre horas del bloque
Operativa: sensor congelado	El valor no cambia en toda la franja	CV: desviación típica / (|media| + 1), próximo a cero

[Tabla 2.3]

El valor esperado es la media histórica de la misma estación, contaminante, bloque y mes, y la dispersión es su desviación típica. Las tres variables son comparables entre estaciones porque no dependen de la escala: una estación de tráfico y una de fondo tienen niveles muy distintos, pero una cobertura de 0,5 o un z-score de 4 significan lo mismo en ambas. El término +1 del coeficiente de variación evita que se dispare cuando la media es cercana a cero.

Se utilizan dos métodos. El de referencia es el z-score: un bloque es anómalo si su valor absoluto supera 3. Es sencillo y fácil de explicar, pero solo mira el nivel, así que no detecta un sensor caído o congelado cuya media sea normal. El detector principal es un Isolation Forest, un algoritmo no supervisado que aísla cada observación mediante particiones aleatorias del espacio de variables: las observaciones anómalas, al ser escasas y diferentes, quedan aisladas con menos particiones que las normales. Se entrena un modelo por contaminante, seis en total, sobre las tres variables de la Tabla 2.3, normalizadas con un escalado propio de cada contaminante. Cada modelo usa 200 árboles, una contaminación del 1 % y una semilla fija para que los resultados sean reproducibles.

Cada bloque recibe tres salidas: una puntuación de anomalía (cuanto mayor, más anómalo), una marca binaria y el valor esperado. Este último permite explicar la marca al usuario en términos concretos, por ejemplo «129,5 µg/m³ frente a 14,9 esperados». Los modelos, los escalados y la lista de variables se guardan juntos en un único fichero, de forma que la inferencia usa exactamente la configuración con la que se entrenó.

Conviene precisar que el parámetro de contaminación fija de antemano que el 1 % de los bloques del histórico se marquen como anómalos. Esa proporción es una decisión de diseño, no un resultado del modelo; lo que aporta el modelo es qué bloques forman ese 1 %.

La elección del Isolation Forest responde a cuatro razones: no necesita etiquetas, combina las tres variables en un único modelo multivariante, se entrena en CPU con scikit-learn, que ya formaba parte del proyecto, y su salida se interpreta a partir de tres variables con significado físico. Se valoraron otras alternativas:

- LSTM Autoencoder. Era el detector principal del diseño inicial y podría captar anomalías en la forma del perfil horario. Se descartó como modelo principal porque requiere GPU y PyTorch y, sin etiquetas, su mejora frente al Isolation Forest no se podría medir. Queda como trabajo futuro.
- Prophet con análisis de residuos. Exige un modelo por serie temporal y solo detecta desviaciones de nivel, no fallos operativos.
- Autoencoder lineal basado en PCA. Es una alternativa de bajo coste con scikit-learn para detectar anomalías de forma, que queda como mejora opcional.

2.6 Predicción

El sistema incorpora un segundo modelo de aprendizaje automático para predecir, a corto plazo, la concentración media de cada contaminante por estación y franja del día. Las estimaciones cubren el día en curso y el siguiente, lo que permite al asistente responder preguntas como «¿cómo estará el aire mañana por la mañana?». En el diseño inicial, estas preguntas recibían siempre una respuesta fija indicando que no podían resolverse.

Planteamiento. Se entrena un modelo por contaminante, siguiendo el mismo criterio que en el detector de anomalías. Se utiliza HistGradientBoostingRegressor, de scikit-learn, por tres motivos prácticos: admite valores ausentes sin necesidad de rellenarlos, no requiere escalar

los datos y ya estaba incluido entre las dependencias del proyecto. También se valoraron otras opciones. LightGBM añadía una dependencia sin aportar una mejora apreciable para este volumen de datos; SARIMA habría obligado a mantener unas 450 series con dificultades para gestionar los huecos; y un LSTM suponía incorporar PyTorch sin contar con datos meteorológicos que justificaran esa complejidad.

Variables. Cada fila recoge los datos de una combinación de estación, contaminante y franja en un día concreto. El objetivo es predecir la media de esa misma franja uno o dos días después. Se utilizan 16 variables: la media del día, los valores de los seis días anteriores para la misma franja, las medias móviles de 3 y 7 días, el valor habitual del mes que se quiere predecir, la cobertura, la franja, la estación y los datos de calendario de la fecha objetivo día de la semana, mes y fin de semana. Las series incluyen todos los días del calendario, aunque falten mediciones. Así, cuando no hay datos de un día, el hueco se conserva y no se sustituye por una observación más antigua.

Referencias de comparación. Para comprobar si el modelo aporta una mejora, sus predicciones se comparan con tres alternativas sencillas: la persistencia, que supone que mañana será como hoy; la climatología, que utiliza el valor habitual del mes calculado con los datos de entrenamiento; y la persistencia semanal, que toma como referencia el mismo día de la semana anterior. Antes de evaluar los resultados se fijó el criterio de éxito: superar la persistencia en el resultado global y en al menos cuatro de los seis contaminantes, y mejorar la climatología en todos ellos.

Prevención de fugas de información. La evaluación respeta el orden temporal de los datos. El modelo se entrena con la información disponible hasta abril de 2025 y se evalúa sobre el año completo siguiente. Los valores habituales de cada mes se calculan únicamente con el periodo de entrenamiento. Se descartaron los valores precalculados del histórico porque incluían todos los años, también el de prueba. Además, se eliminan del entrenamiento las filas cuya fecha objetivo cae dentro del periodo de prueba. La configuración del modelo se elige mediante una validación temporal interna, sin utilizar el conjunto de prueba para tomar esa decisión.

Dos horizontes de predicción. Como los datos llegan hasta el último día completo, predecir un día por delante suele equivaler a predecir hoy. Para cubrir también mañana, se entrena un segundo modelo a dos días vista, con el mismo diseño y las mismas variables. Los hiperparámetros de cada horizonte se ajustan por separado mediante validación temporal interna. El servicio elige el horizonte tomando como referencia el último día con las cuatro franjas cargadas. Si la fecha solicitada ya tiene mediciones, deriva la petición a la consulta de datos; si queda más allá de los dos días de alcance, informa de que no puede ofrecer una predicción.

2.7 Asistente conversacional

El asistente es un servicio propio, el agente, con un bucle de herramientas escrito a mano. LlamaIndex se usa solo como cliente del LLM en Amazon Bedrock. Cada turno empieza con

una llamada corta al modelo, a temperatura 0, que actúa de clasificador. Devuelve una o dos intenciones (documental, datos, predicción, charla o fuera de alcance) y el tema de la pregunta (salud, normativa, proyecto o ninguno). A partir de ahí decide el código, no el modelo, con una tabla fija que indica qué herramientas se ofrecen en el turno, cuáles son obligatorias y con qué prompt se responde. Una pregunta fuera de alcance recibe una frase fija sin volver a llamar al modelo, y una charla se responde sin herramientas. Si el clasificador falla o su respuesta no tiene el formato esperado, el turno sigue con un valor seguro que ofrece todas las herramientas: un clasificador caído no debe dejar al usuario sin respuesta.

El agente dispone de tres herramientas: búsqueda documental en el servicio RAG, consulta de mediciones y predicción. El RAG y el servicio de predicción publican ellos mismos la definición de su herramienta, de modo que el agente no guarda una copia que pueda quedar desfasada. Una herramienta que no está permitida en el turno no se le ofrece al modelo, y si el modelo la pide igualmente recibe un error y no se ejecuta. Si una herramienta es obligatoria y el modelo no la usa, la lanza el código con la pregunta original.

Las fechas de cada consulta, que fueron un problema al principio del desarrollo, se calculan en Python a partir de la pregunta para periodos habituales (semana pasada, año pasado..), evitando que las elija el LLM, ya que provocaba errores.

El bucle admite como mucho tres llamadas con herramientas; después, una síntesis sin herramientas redacta la respuesta a partir de los resultados obtenidos. En las preguntas documentales, el modelo devuelve sus afirmaciones en un formato estructurado que el RAG valida antes de entregarlas al usuario. Las preguntas mixtas se responden en dos bloques, datos y documentación, para que las cifras de la base de datos no se mezclen con las citas del corpus.

Para la consulta de mediciones se construyeron dos variantes, que se eligen por configuración sin cambiar el resto del agente. En el SQL libre, un LLM redacta la consulta sobre la vista mediciones_bloques; un validador comprueba que es una única sentencia de lectura sobre esa vista e impone un límite de filas, y PostgreSQL la ejecuta con el usuario de solo lectura. En el catálogo cerrado, el LLM solo elige una de siete consultas predefinidas y sus parámetros, y el código construye el SQL. Así, reglas como la media ponderada por horas pasan del prompt al código. En un lote de preguntas no usado para ajustar el sistema, el SQL libre respondió más preguntas, porque resuelve las que no encajan en ninguna de las siete consultas. En las preguntas que sí encajan, las dos variantes practicamente empataron, y el catálogo protegió mejor con los modelos más débiles (apartado 3.4).

En producción se usa el SQL libre.

2.8 Frontend

La interfaz está desarrollada con React 19, TypeScript y Vite, y permite navegar entre varias pantallas sin recargar la página. Se sirve mediante nginx, que también redirige las peticiones

a la API del backend, evitando configurar CORS o incluir direcciones del backend en el frontend. Inicialmente se planteó usar Streamlit, pero se optó por una aplicación propia para controlar mejor cómo se muestran e interpretan los datos.

Contrato con el backend. Los tipos de TypeScript se generan a partir del esquema OpenAPI de la API. La integración continua comprueba que coincidan con la versión guardada en el proyecto, de modo que los cambios incompatibles se detecten durante la compilación del pull request.

Pantallas. La aplicación incluye una landing page, una vista de estadísticas con las anomalías y los bloques marcados del último día con datos, la tendencia de los últimos siete días y un mapa de las 24 estaciones construido con Leaflet. El mapa permite consultar cada estación, ver sus series por franjas, seleccionar fechas y comparar hasta tres estaciones.

El asistente está disponible en toda la aplicación mediante una burbuja flotante. Las respuestas de texto libre aparecen progresivamente, mientras que las que requieren validación, como las documentales, se muestran completas una vez comprobadas. Las respuestas documentales incluyen sus fuentes y un aviso médico.

2.9 Observabilidad y métricas

El sistema se vigila en tres niveles, cada uno con la herramienta que mejor encaja: las trazas del agente, para entender cómo se resuelve cada pregunta; las métricas de uso del asistente, para saber si sirve a quien lo usa; y las métricas de infraestructura, para detectar fallos y vigilar el coste. Las trazas se registran con OpenTelemetry y se guardan en Phoenix, una herramienta de código abierto para observar aplicaciones con LLM que se ejecuta como un contenedor más en la EC2. Cada turno deja un árbol con la clasificación, cada llamada al modelo con sus mensajes y tokens, cada herramienta con su resultado y las decisiones que tomó el código, como una búsqueda forzada o una herramienta vetada. Las trazas fueron la principal fuente para depurar el agente y comparar modelos. Phoenix las conserva 30 días y está abierto para la evaluación del tribunal .

Con esas trazas, el propio agente calcula los indicadores de uso del asistente, y la web los muestra en una página de administración: preguntas por día, reparto por intención, ruta y tema, tasa de resolución, coste por pregunta, latencia y tiempo hasta el primer token, fallos de herramienta y satisfacción. La satisfacción sale de una valoración positiva o negativa que la web pide al usuario. El voto se guarda en Phoenix junto a la traza del turno valorado, sin necesidad de una tabla nueva. La página solo muestra agregados, nunca el texto de las preguntas ni de las respuestas. Esta medida complementa la evaluación con baterías de preguntas (apartado 3.4): las baterías miden lo que pregunta el equipo, y estos indicadores, lo que preguntan los usuarios reales.

Las métricas de infraestructura las recoge Amazon CloudWatch, que reúne los registros de los servicios y las métricas de la Lambda, la EC2, la base de datos y Bedrock. Once alarmas avisan por correo de errores de la función, caídas o falta de memoria y disco en el servidor,

errores de la web, falta de espacio en la base de datos y errores o consumo excesivo de tokens del modelo de lenguaje. Además, cada ejecución del pipeline publica sus métricas de negocio en el propio registro: horas cargadas, estaciones con datos y porcentaje de bloques anómalos. Así, una alarma avisa si pasa un día sin datos aunque la función no haya fallado. 9 de las alarmas, métricas y panel de control se definen como código con CloudFormation.

Acceso: Los indicadores de uso son públicos en /admin/uso (enlace «Administración» del pie). Phoenix se sirve en /phoenix/ a través de nginx. El panel jupiter y las alarmas se consultan en la consola de CloudWatch de la cuenta del proyecto.

2.10 Infraestructura en AWS y CI/CD

El sistema se despliega en una cuenta de AWS del máster, en la región de Irlanda (eu-west-1), con cuatro criterios: pagar por uso, preferir servicios gestionados, dar a cada componente solo los permisos que necesita y no guardar claves de acceso permanentes. La Tabla 2.4 resume los servicios utilizados.

Servicio	Uso en el proyecto
AWS Lambda	Pipeline de ingesta, empaquetado como imagen de contenedor
Amazon EventBridge Scheduler	Ejecución programada del pipeline y encendido y apagado del entorno en la fase de desarrollo
Amazon RDS for PostgreSQL	Base de datos (PostgreSQL 18)
Amazon S3	Histórico, conjuntos procesados, modelos y ficheros de arranque, con versionado
Amazon ECR	Registro de las imágenes de contenedor
Amazon EC2	Web, API, agente, servicio RAG, servicio predicción y Phoenix en contenedores Docker (instancia t3.medium)
Amazon Bedrock	Modelos de lenguaje del asistente
AWS Systems Manager	Secretos y configuración (Parameter Store), despliegue en la EC2 (Run Command) y acceso sin SSH (Session Manager)
Amazon CloudWatch y SNS	Registros, métricas, alarmas y panel de control
AWS IAM	Roles con permisos mínimos para cada componente
AWS CloudFormation	Alarmas, métricas y panel definidos como código.

[Tabla 2.4]

Ejecución. El pipeline se ejecuta en Lambda, que solo cobra por los segundos de ejecución, y guarda los datos en RDS, que gestiona las copias de seguridad y los parches. La web, la API, el agente, el RAG, el servicio de predicción y Phoenix se ejecutan en una instancia EC2, cada uno en su propio contenedor. La máquina descarga de ECR la versión activa de cada servicio, de modo que cada pieza se actualiza o se revierte por separado.

Seguridad. En la EC2 solo nginx expone un puerto. La base de datos es accesible desde internet porque la Lambda necesita salir a internet para llamar a la API municipal, pero exige conexiones cifradas y su contraseña se guarda cifrada en Systems Manager. El agente usa Bedrock con el rol de la máquina, sin claves, y lee la base de datos con un usuario de solo lectura. Como el chat es público y cada pregunta se paga, el gasto en Bedrock se vigila con un presupuesto diario y una alarma de consumo.

Integración y despliegue continuos. Cada pull request ejecuta las pruebas en GitHub Actions. El despliegue se lanza a mano eligiendo el componente: repite las pruebas, construye la imagen, la sube a ECR y actualiza solo esa pieza, comprobando que arranca correctamente. Otro workflow revierte cualquier componente a una versión anterior en segundos. GitHub accede a AWS con credenciales temporales mediante OpenID Connect, sin claves guardadas en el repositorio.

2.11 Stack tecnológico

La Tabla 2.5 resume las tecnologías utilizadas en cada capa del sistema.

Capa	Tecnologías
Lenguajes	Python, TypeScript, SQL
Análisis de datos	pandas, NumPy, pyarrow (Parquet), matplotlib, seaborn, Jupyter
Base de datos	PostgreSQL 18, SQLAlchemy, psycopg
Aprendizaje automático	scikit-learn (Isolation Forest, HistGradientBoosting), joblib
RAG	sentence-transformers (multilingual-e5-base), ChromaDB
Agente LLM	Amazon Bedrock, LlamaIndex (cliente del LLM), sqlglot (validación de SQL), OpenTelemetry y Arize Phoenix (observabilidad)
Servicios backend	FastAPI, Uvicorn, Pydantic, httpx
Frontend	React 19, TypeScript, Vite, React Router, TanStack Query, Recharts, Leaflet, openapi-typescript, nginx
Contenedores	Docker, Docker Compose

Capa	Tecnologías
Nube (AWS)	Lambda, EventBridge Scheduler, RDS, S3, ECR, EC2, Bedrock, Systems Manager, CloudWatch, SNS, IAM, Budgets y CloudFormation
CI/CD y calidad	GitHub, GitHub Actions, pytest, Vitest, Make

[Tabla 2.5]

Capítulo 3 - Resultados

Este capítulo presenta las medidas obtenidas para cada componente. Como no se dispone de datos etiquetados para todos ellos, cada apartado indica con qué se compara: un método de referencia, casos de evaluación construidos para el proyecto o eventos reales conocidos.

3.1 Detección de anomalías

Para valorar el detector se comparó con el método de referencia, el z-score, sobre los más de 1,2 millones de bloques del histórico. El z-score marca el 1,35 % de los bloques y el Isolation Forest, el 1 %, la proporción que se le fija al configurarlo. Lo llamativo es que casi nunca coinciden: solo 2.198 bloques los marcan los dos.

Cada método encuentra un tipo de problema distinto:

- El Isolation Forest detecta sensores que fallan. A tres de cada cuatro bloques que marca les falta al menos un tercio de las horas. El 16 de septiembre de 2026, primer día en la nube, marcó 15 bloques y todos eran estaciones que habían dejado de medir; el z-score no marcó ninguno, porque el nivel era normal.
- El z-score detecta episodios de contaminación. Durante la calima del 15 de marzo de 2022 (apartado 2.3), marcó los 52 bloques de PM10 del día; el Isolation Forest solo marcó 7. El modelo trabaja con una cuota fija del 1 %, y esa cuota la ocupan sobre todo los sensores averiados.

	z-score: normal	**z-score: anómalo**
Isolation Forest: normal	1.246.843	15.053
Isolation Forest: anómalo	10.550	2.198

[Tabla 3.1]

3.2 Predicción

El modelo de *gradient boosting*, entrenado por separado para cada contaminante (apartado 2.6), se evaluó con un año completo de datos no utilizados en el entrenamiento. Se entrenó con los bloques de enero de 2018 a abril de 2025 —1,1 millones de filas— y se evaluó sobre

los doce meses siguientes, con 147.790 predicciones comparables. Se aplicaron las medidas contra fugas de información descritas en el apartado 2.6 y se comparó con las tres referencias mediante el error absoluto medio (MAE) y el *skill*, que expresa la mejora relativa del MAE respecto a la persistencia.

Método	MAE a un día vista (µg/m³)
Modelo	10,11
Persistencia	10,46
Persistencia semanal	13,38
Climatología mensual	15,00

[Tabla 3.2. Error global de predicción a un día vista sobre el año de prueba.]

A un día vista, el modelo supera ligeramente a la persistencia, con un *skill* global de +0,03. La mejora es limitada porque la contaminación de mañana suele parecerse a la de hoy. Aun así, mejora en cinco de los seis contaminantes: O_3 (+0,12), NO_2, PM10 y PM2.5 (+0,08) y NOx (+0,01). Solo queda por debajo en NO (−0,18), mientras que supera a la climatología en los seis. Por franjas, la mayor mejora aparece por la mañana (+0,12), coincidiendo con la hora punta de tráfico; por la tarde y la noche, el resultado es similar al de la persistencia.

A dos días vista, el horizonte que permite responder sobre mañana, la ventaja aumenta: el MAE es de 11,86 frente a 12,80 de la persistencia, con un *skill* de +0,07. Se mantiene la mejora frente a la persistencia en cinco contaminantes y frente a la climatología en los seis. Al ampliar el horizonte, el error de la persistencia crece un 22 %, frente al 17 % del modelo. Esto sugiere que el modelo aprende patrones que van más allá de repetir el último valor disponible.

Los resultados también muestran dos limitaciones. La primera son los picos nocturnos extremos de NOx, de entre 400 y 580 µg/m³, que ni el modelo ni las referencias consiguen anticipar. Su predicción requeriría información meteorológica que el sistema no tiene. La segunda es la cantidad de datos: una versión inicial, entrenada con solo cuatro meses de ingesta propia, quedaba por debajo de la persistencia (*skill* −0,28); con ocho años de histórico, el mismo diseño consigue superarla. Queda como trabajo futuro estimar la probabilidad de superar un umbral, lo que permitiría ofrecer avisos de episodios de contaminación más útiles para el usuario.

3.3 Recuperación del RAG

Para medir si el sistema encuentra la información correcta se preparó un conjunto fijo de 30 preguntas, cuyo tipo se decidió antes de ver ningún resultado: 15 que el corpus sí responde, al menos una por documento; 10 ajenas al dominio, siete de ellas cercanas a él y tres sin relación alguna, como la capital de Francia; y 5 preguntas trampa, que parecen del corpus pero no tienen respuesta en él.

En 14 de las 15 preguntas documentales, la sección que contiene la respuesta aparece entre los cuatro fragmentos que se entregan al LLM. El único fallo es comprensible: ante una pregunta sobre el monóxido de nitrógeno (NO), el buscador prioriza los textos sobre el dióxido (NO_2), porque apenas distingue ambos términos.

El resultado más relevante es el de las preguntas ajenas. El sistema solo entrega fragmentos al LLM si se parecen lo suficiente a la pregunta, y ese límite es el umbral de evidencia. Con el primer umbral, fijado a ojo con preguntas muy alejadas del dominio, se rechazaban solo 3 de las 10 ajenas. Las siete cercanas, sobre tiempo, tráfico o transporte, recibían fragmentos y el LLM podía acabar respondiendo sobre algo que el corpus no cubre.

Al recalibrar el umbral con estas preguntas se rechazan las 10, a cambio de perder una documental (13 de 15 en lugar de 14): la que pregunta si el asistente puede hacer un diagnóstico médico. Se aceptó ese coste por un criterio de producto: en un asistente que habla de salud, es preferible que se abstenga de más a que responda fuera de su ámbito.

Las cinco preguntas trampa, en cambio, no se pueden filtrar así, porque el buscador las sitúa tan cerca del corpus como las preguntas legítimas. En ellas la defensa es otra: el LLM debe declarar que no tiene evidencia suficiente, y el backend comprueba que cada cita corresponde a un fragmento real del corpus (apartado 2.7). Además, el umbral se calibró y se midió con las mismas 30 preguntas, así que las cifras describen este conjunto y no garantizan el mismo comportamiento con preguntas nuevas.

3.4 Evaluación del asistente

Para elegir los modelos se prepararon baterías de preguntas con su respuesta esperada, que en las preguntas de datos se calculó en PostgreSQL, y se ejecutaron con el código del agente contra Amazon Bedrock. Unas sirvieron para ajustar el sistema: 31 preguntas para el clasificador, 12 preguntas sueltas, 5 conversaciones con 11 turnos y 16 preguntas de datos. Otras se reservaron como control y no se usaron para ajustar nada: 28 preguntas de datos nuevas y 10 turnos de predicción. Cada batería se ejecutó una sola vez por modelo, así que una diferencia de uno o dos casos no es significativa.

Clasificador. Tras ajustar sus instrucciones, gpt-oss-120b acierta las 31 preguntas iniciales, pero falla más con preguntas nuevas.

Preguntas de datos. Con el SQL libre, la mejor combinación, gpt-oss-120b para la conversación y para redactar el SQL, pasó de 4 a 8 aciertos de 11 en cuatro rondas de ajuste, gracias a reglas en código y no a un cambio de modelo. Como el catálogo se diseñó con esas mismas preguntas, las dos variantes se compararon en la batería de control. Con gpt-oss-120b, el SQL libre acertó 15 de 20 preguntas y el catálogo, 12: la diferencia viene de las 6 que no encajan en ninguna de las siete consultas, porque en las 14 restantes hubo un empate técnico (10 frente a 11). Con Ministral 14B, el catálogo acertó más en esas 14 (9 frente a 7), porque no le deja inventar contaminantes ni periodos. Ninguna combinación

resolvió las preguntas mixtas: la parte documental se abstenía o contradecía a la de datos. En producción se usa el SQL libre.

Comparación de modelos. A Ministral 14B y gpt-oss-120b, los dos candidatos iniciales, se sumaron después Qwen3 Next 80B y Nemotron 3 Super

	Ministral 14B	**gpt-oss-120b**	**Qwen3 Next 80B**	**Nemotron 3 Super**
Clasificador (31) [1]	28	**31**	29	31
SQL libre (20) [2]	10	15	**16**	**16**
Catálogo (20) [2]	9	12	10	**14**
Predicción (8) [3]	7	**8**	7	7
Latencia mediana	8,3 s	7,5 s	7,9 s	**4,9 s**
Coste 12 preguntas	0,0059 $	0,0087 $	0,0072 $	0,0078 $

[Tabla 3.3 Coste de pruebas de LLMs.]

(1) Hubo otra pasada con 29. (2) Con Ministral, el SQL lo redactó Qwen3 Coder. (3) Sin contar dos casos que fallaron en los cuatro modelos por el entorno de prueba.

Para la conversación y el SQL se eligió Nemotron 3 Super, el que más acierta en la batería de control y el más rápido, con un coste similar a los demás. El clasificador se mantiene en gpt-oss-120b, el único que acertó las 31 preguntas en todas las pasadas; los dos fallos de Nemotron fueron preguntas de predicción, justo la intención que lleva a la nueva herramienta.

Uso en la web. Hasta el 25 de octubre, el asistente publicado recibió 174 preguntas en 28 conversaciones, todas del equipo y casi todas respondidas por Ministral 14B, el modelo configurado hasta ese día. Completó sin error 166 de las 174 preguntas (95 %), contando como completadas las ajenas al tema en las que avisa de que no puede ayudar; no se comprobó si cada respuesta era correcta. La mitad de las respuestas llegaron en menos de 6 segundos y 9 de cada 10, en menos de 14. En salud y normativa presentó sus fuentes correctamente a la primera en 68 de 69 casos, y cada pregunta costó de media 0,0013 $.

3.5 Sistema en producción y costes

Ingesta: Entre el 16 de septiembre y el 21 de octubre de 2026, el pipeline se ejecutó los 36 días sin ningún error: 38 ejecuciones, dos de ellas lanzadas a mano, con una duración media de 8,5 segundos. En ese tiempo cargó 92.400 mediciones horarias, el 93,4 % de ellas válidas, y 15.336 bloques nuevos, unos 428 al día (107 combinaciones de estación y contaminante por cuatro bloques).

Servicio	**16–30 sep.**	**1–22 oct.**
Base de datos (RDS: instancia, disco y copias)	7,10 $	4,67 $
Direcciones IP públicas	1,72 $	2,64 $
Modelos de lenguaje (Bedrock)	—	1,49 $
Servidor de la aplicación (EC2: instancia y disco)	—	1,17 $
Otros (ECR, S3, Systems Manager)	0,03 $	0,37 $
Total	**8,85 $**	**10,35 $**

[Tabla 3.4]

De estas cifras se extraen tres conclusiones:

- La base de datos es el mayor gasto. Encenderla solo por la noche, desde octubre, redujo su uso de 24 a unas 5 horas diarias, pero el disco y las copias de seguridad se pagan igualmente.
- Las direcciones IP públicas cuestan más de lo previsto: unos 0,005 $ por hora cada una, y la del servidor se cobra también con la máquina apagada. En octubre supusieron una cuarta parte de la factura y no figuraban en la estimación inicial.
- El modelo de lenguaje es barato a esta escala: 1,49 $ en octubre, incluidas las campañas de evaluación de modelos.

Para la entrega, la base de datos y el servidor quedan encendidos de forma continua, lo que eleva el coste estimado a unos 56 $ al mes más el uso de Bedrock: unos 24 $ de la base de datos y las direcciones IP públicas, y unos 32 $ del servidor.